Swinburne University of Technology
School of Science, Computing, and Emerging Technologies

SWE40006 - Software Deployment and Evolution

Deployment Portfolio Task 3

Student name: Ngoc Van Nhi Do
Student ID: 105551765
Submission date: DON'T FORGET THISSSSSSSSSSS

Deployment Portfolio Task 3

1 Declared Target Level

All four subtasks 3.1, 3.2, 3.3, and 3.4 were attempted.

Table 1 below contain relevant links as proof of successful task execution.

Table 1: Publicly accessible URLs for live grading verification

Source	URL
ALB DNS Name	http://task-3-3-alb-2099008439.ap-southeast-2.elb.amazonaws.com/

2 Execution Evidence

2.1 Task 3.1

For this task, I deployed a WordPress application onto an Amazon EC2 instance.

2.1.1 AWS Environment Setup

First, I created a new AWS account on the free plan, which gave me $100 in free credits.

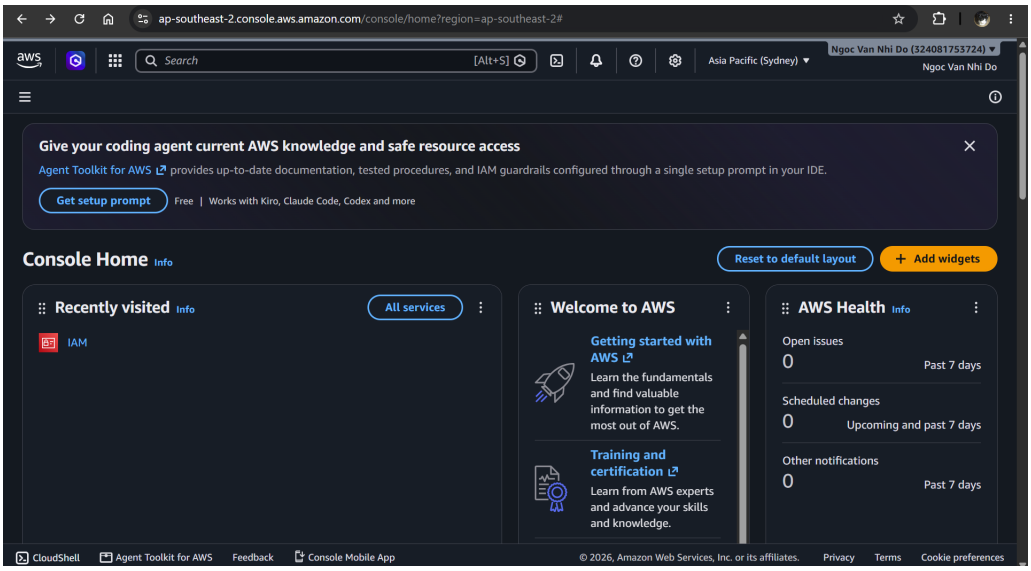

Figure 1: The AWS Console interface after I had successfully signed up.

I then created a new IAM user for my coursework named `SWE40006_student` rather than using the root account. This user is assigned to a new user group `EC2-Lab-Users` which is given full access to EC2 resources (using `AmazonEC2FullAccess`).

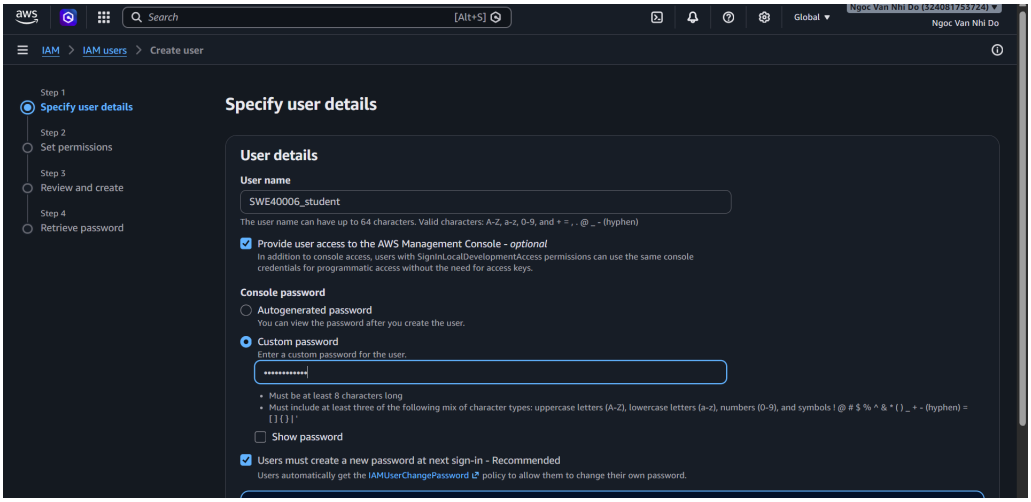

Figure 2: Creating a new IAM user `SWE40006_student`.

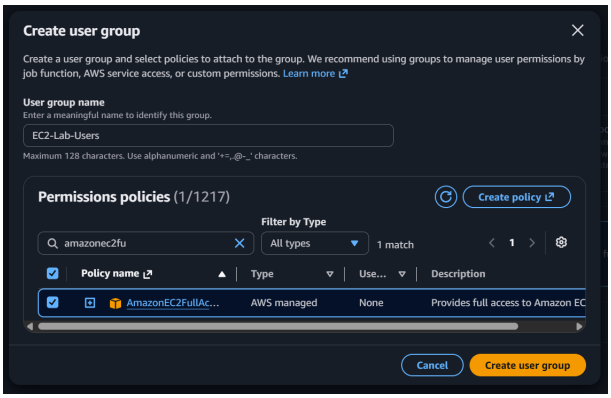

Figure 3: Adding `AmazonEC2FullAccess` to the user group `EC2-Lab-Users`, which is then assigned to `SWE40006_student`.

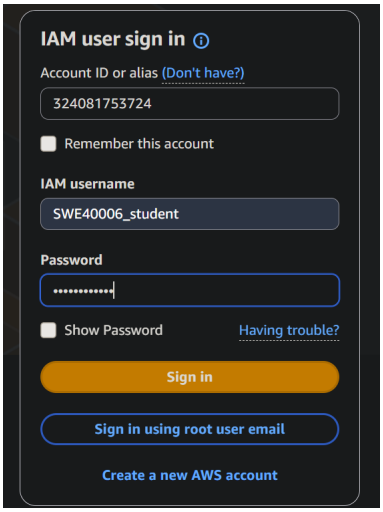

Figure 4: Logging into the AWS Console as the new IAM user.

To securely login into my EC2 instances, I must generate an SSH key pair. The private key was downloaded and stored on my local machine for later authentication.

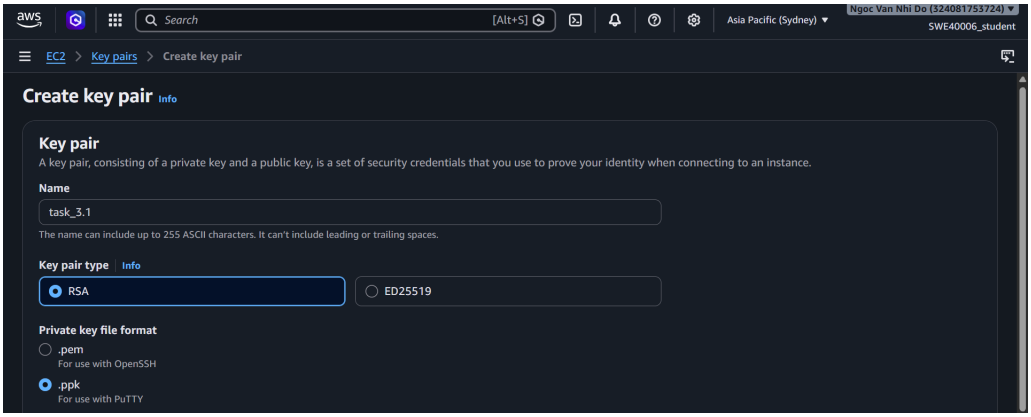

Figure 5: Creating a new key pair `task-3.1`.

2.1.2 Compute Provisioning

I launched a new EC2 instance called `Task 3.1`. This instance runs Amazon Linux 2023, uses a `t3.micro` instance time to remain eligible for the free tier, and is configured with the previously generated key pair.

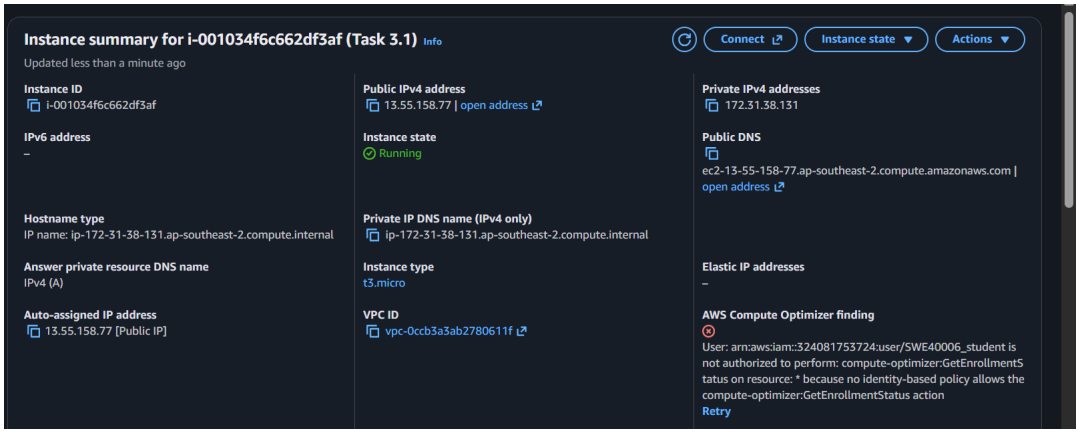

Figure 6: Creating a new Amazon EC2 instance.

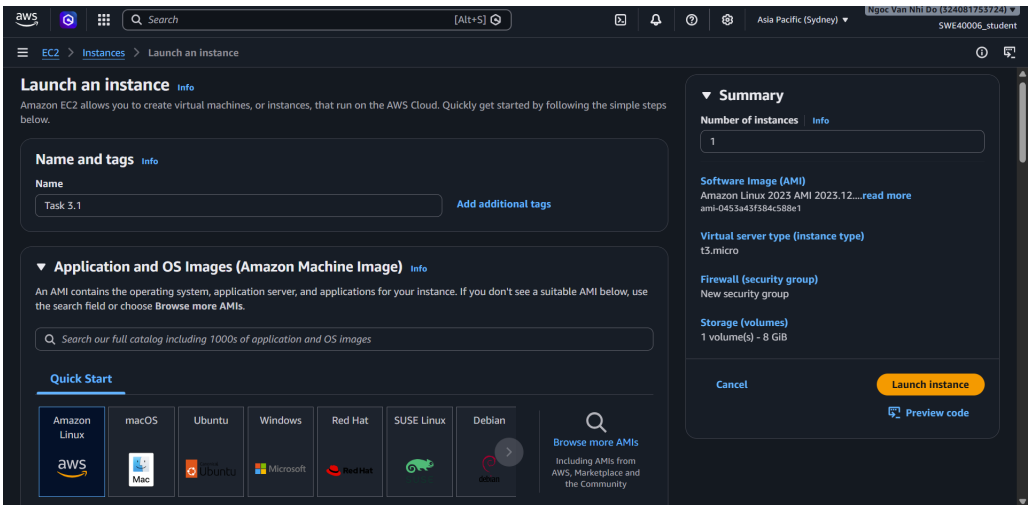

Figure 7: Amazon Linux 2023 was chosen as the OS for this instance.

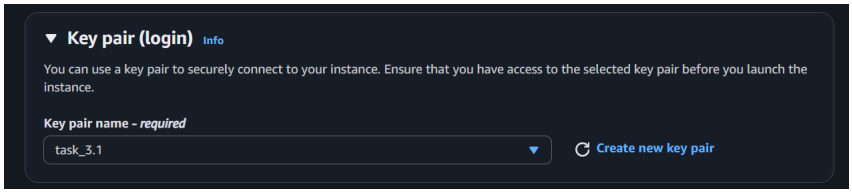

Figure 8: Configuring the instance with the `task-3.1` key pair.

I then configured the instance with a new security group `task-3.1-sg`, which allowed SSH (22), HTTP (80), and HTTPS (443) inbound traffic from anywhere.

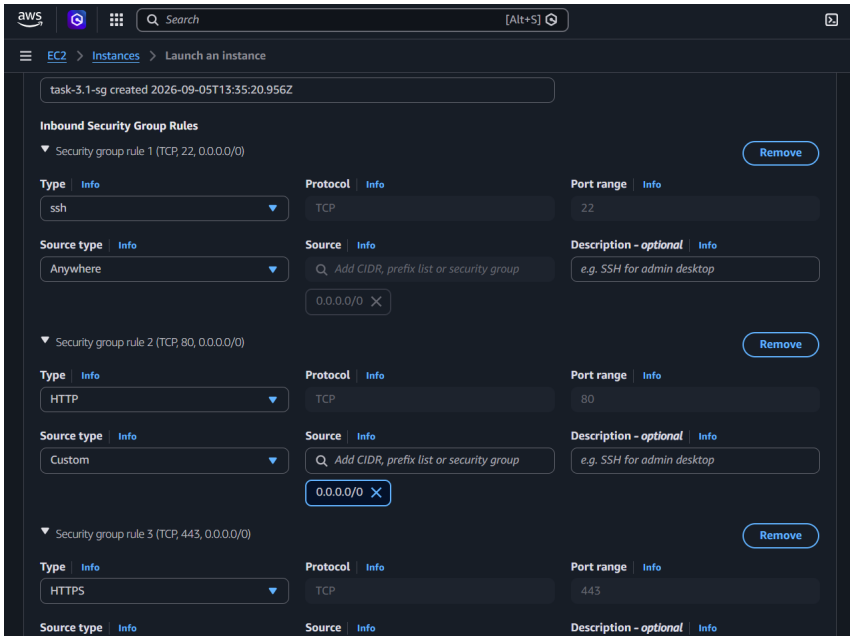

Figure 9: Configuring the security rules to allow inbound SSH, HTTP, and HTTPS traffic to this instance.

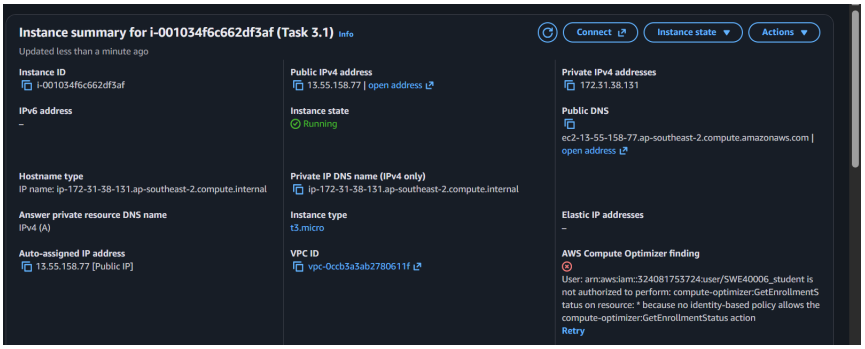

Figure 10: The new instance is successfully created.

I then SSH-ed to the instance using PuTTY.

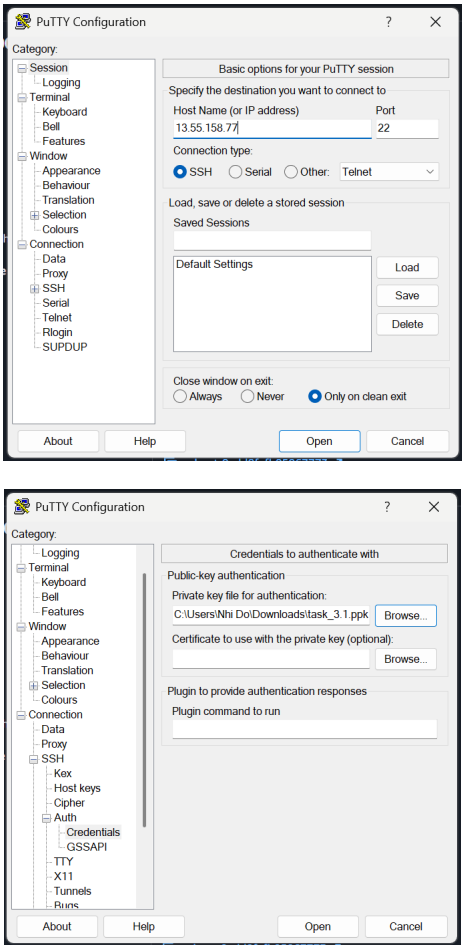

Figure 11: Connecting to the EC2 instance using SSH.

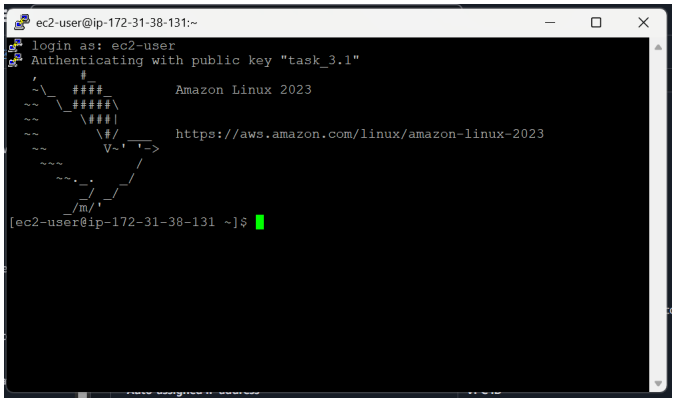

Figure 12: Connection was successful.

I then installed the Apache web server and started it so that it could later host my web application.

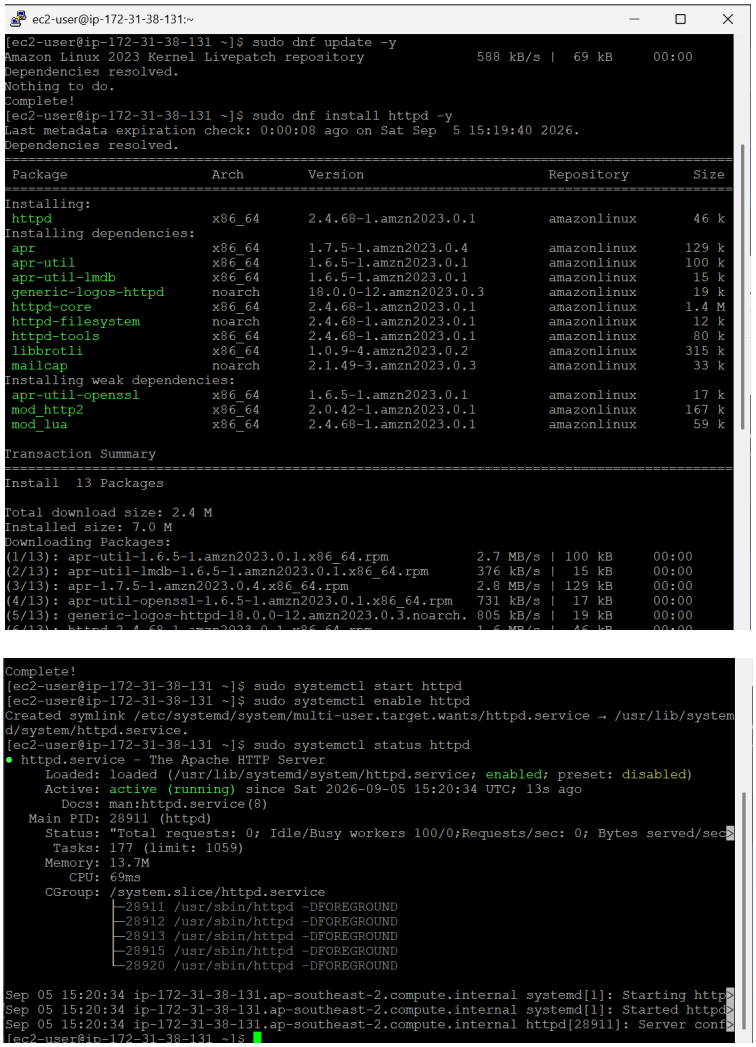

Figure 13: Installing and starting the web server.

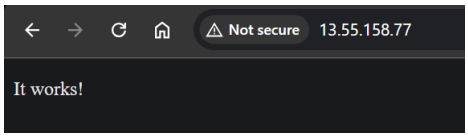

Figure 14: Verifying that the web server is publicly accessible.

2.1.3 Application Deployment

Before deploying the WordPress application, I needed to install its dependencies which included installing PHP and MariaDB.

Figure 15: Installing PHP.

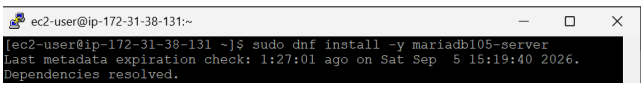

Figure 16: Installing MariaDB.

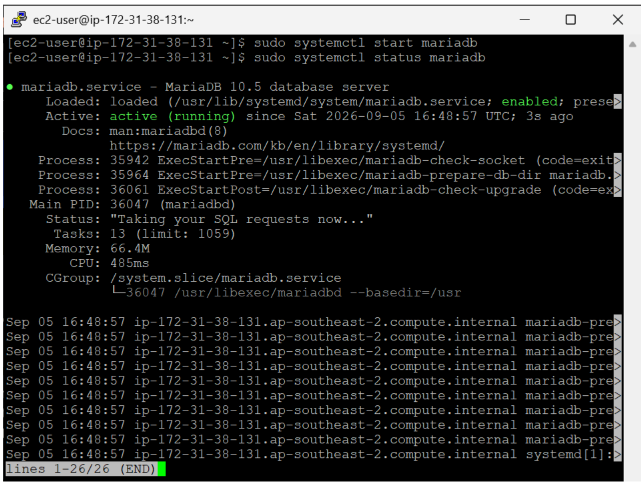

Figure 17: Starting and enabling the MariaDB service.

I then set up a local database for the web application.

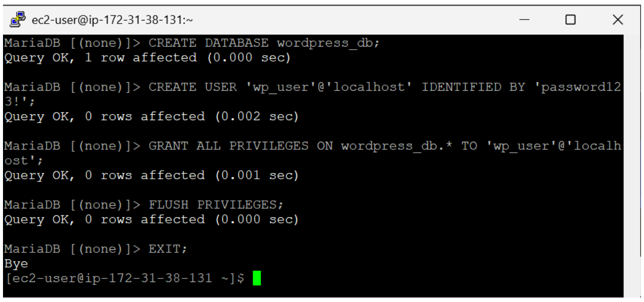

Figure 18: Setting up a local database named `wordpress_db` for the WordPress application.

I installed and extracted the WordPress package. The command `chown` made Apache the owner of the WordPress files, while `chmod` set who could read, write, or access those files. These commands ensured that Apache had the correct permissions to run WordPress.

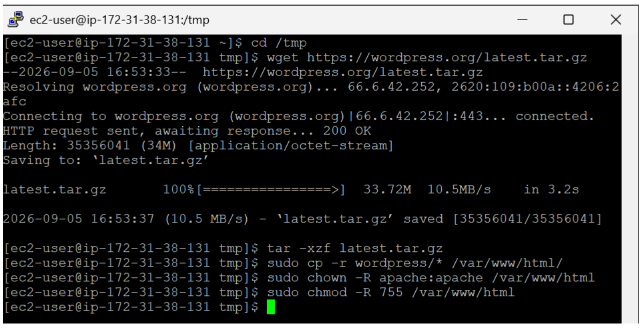

Figure 19: Installing WordPress and configuring permissions.

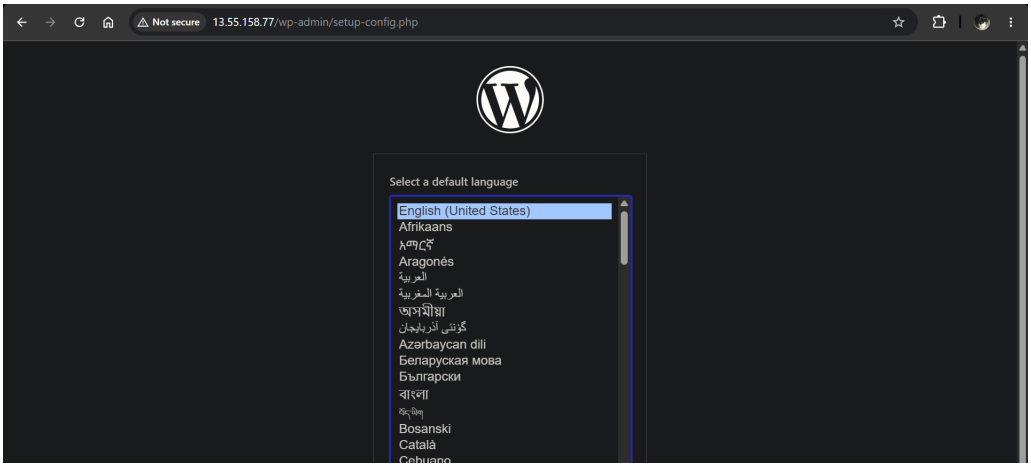

Figure 20: The deployed WordPress application.

2.2 Task 3.2

For this task, I configured my web application to use a database hosted on an Amazon RDS instance, effectively decoupling the database from the EC2 instance. I also created a backup of my web application by storing my files on an S3 bucket, and demonstrated how I could restore my application using this backup.

2.2.1 Database Decoupling

I started by creating a new RDS instance and configured it to use MariaDB as the database engine. I also connected the EC2 instance to this database right away, so that AWS automatically allowed inbound traffic on port 3306 from the web application to the database for me.

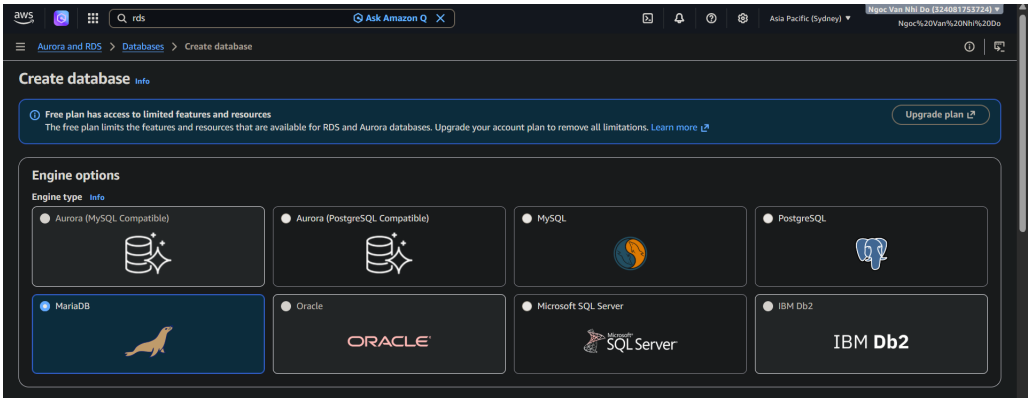

Figure 21: Creating a new RDS instance.

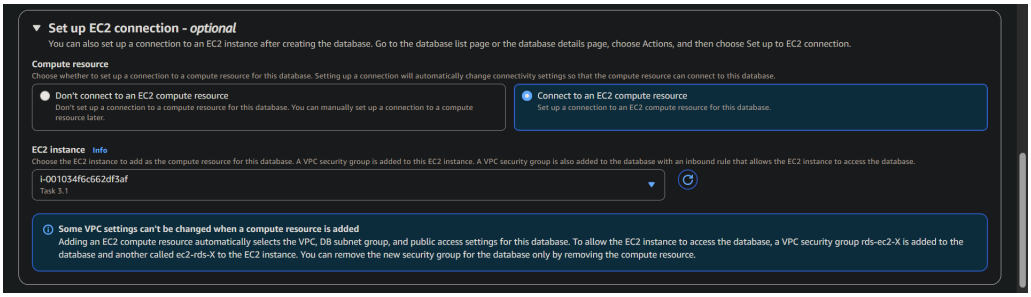

Figure 22: Connecting the EC2 instance from the previous task.

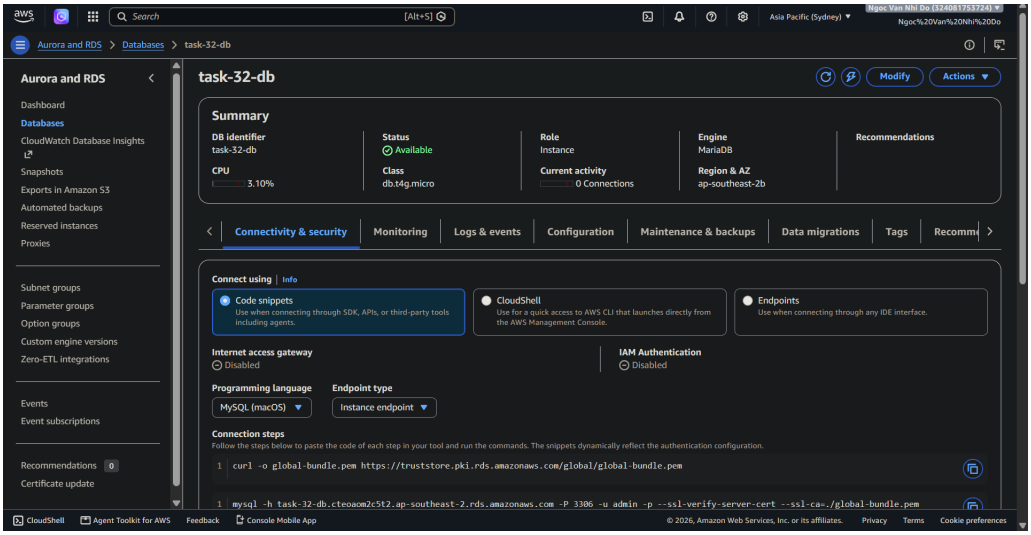

Figure 23: The database is successfully created.

From the EC2 instance's command line, I established a connection to the RDS instance using its endpoint at task-32-db.cteoaom2c5t2.ap-southeast-2.rds.amazonaws.com.

Figure 24: Connecting to the database from the EC2 instance's terminal.

Once I could connect to the database, I created a new database `wordpress_db` to be used by my application. I then exited the DB connection, configured the [wg_config.php] file with the required database information, including the database name, username, and password, along with the other necessary credentials. This allowed the WordPress application to connect to and use the configured database. Finally, I restarted the Apache web server and verified that the application has been successfully configured to use the database. As I had not installed the actual WordPress application by this point, the database displayed all empty entries.

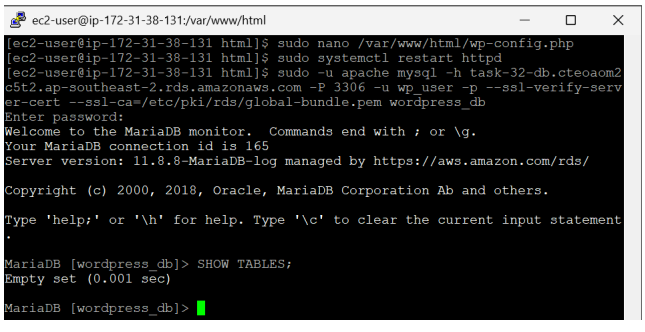

Figure 25: Editing the database configurations and verifiying the connection to the RDS database.

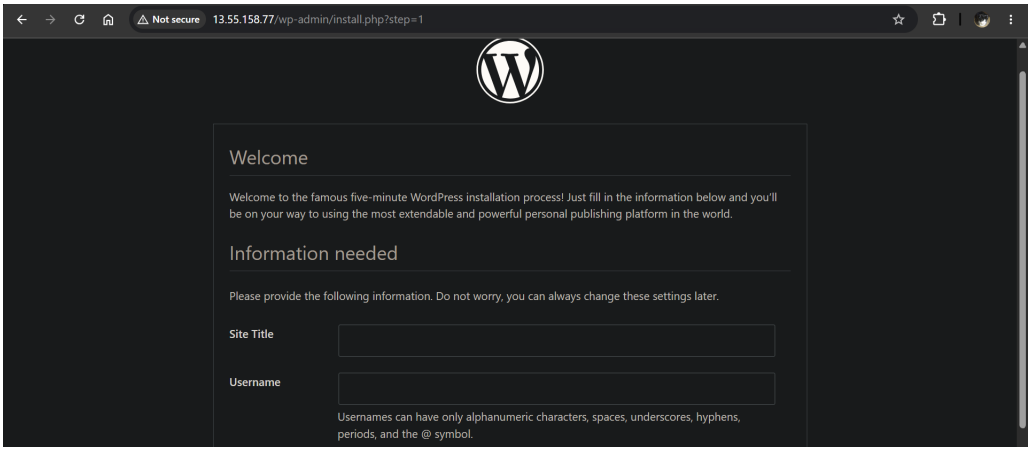

Figure 26: The application now shows the WordPress installation form, which means the database has successfully been connected.

2.2.2 Backup and Restore

I started by creating a new S3 bucket with the name `swe40006-105551765-bucket` to store my backup files.

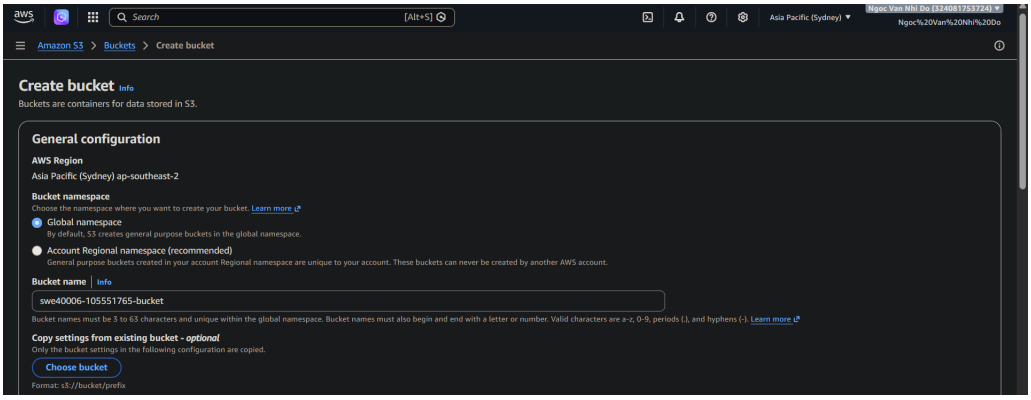

Figure 27: The application now shows the WordPress installation form, which means the database has successfully been connected.

I also created a new IAM role allowing full access to S3. This role is then attached to my EC2 instance so that the server could read and write files to the S3 bucket.

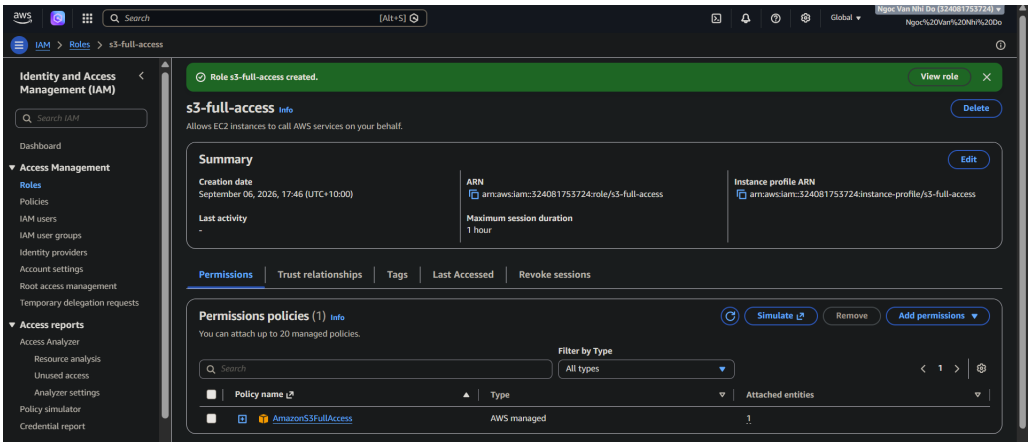

Figure 28: Creating a new IAM role with the `AmazonS3FullAccess` policy.

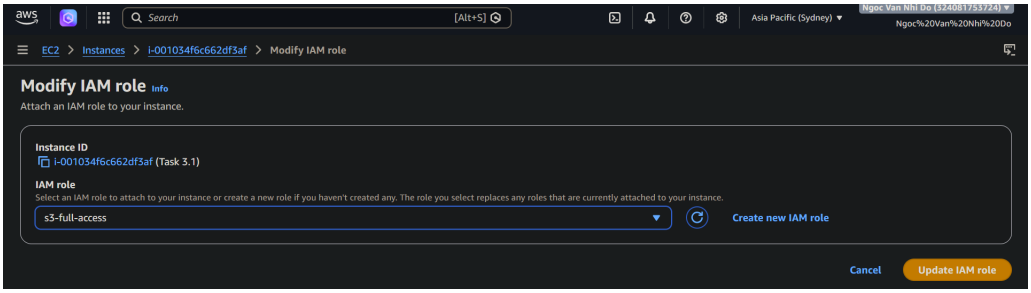

Figure 29: Adding the role to my EC2 instance.

I then made a backup of my files by archiving them into a `.tar` file to avoid having to individually handle hundreds of different files. This archive is then uploaded to the S3 bucket, serving as the backup to my web server.

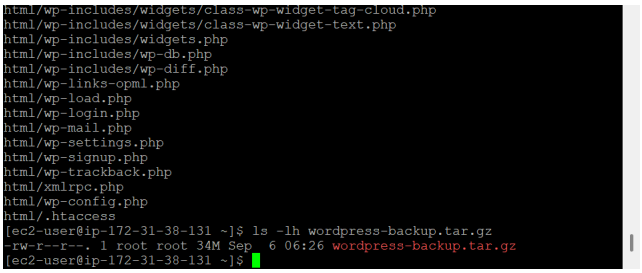

Figure 30: Creating the backup as a `.tar` archive.

Figure 31: Uploading the backup to the S3 bucket.

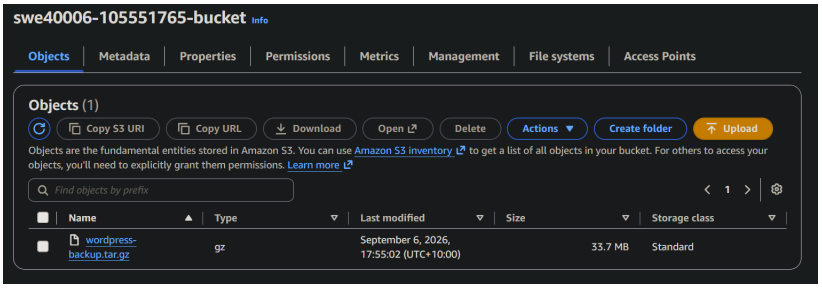

Figure 32: I could see that the backup has been uploaded using the AWS Console.

To demonstrate that I can restore my web server, I first removed all my web application files to simulate a deletion.

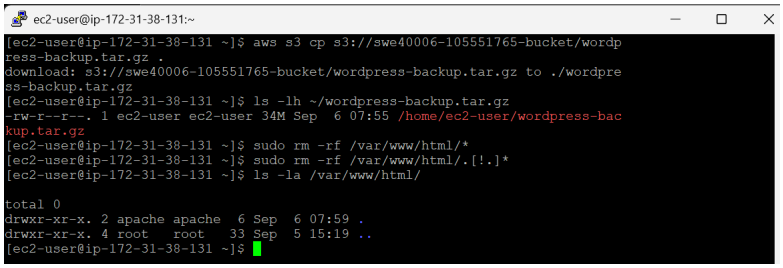

Figure 33: Removing all web application files on my EC2 instance.

Once the files were removed, accessing my website displayed only an "It works!" message rather than the WordPress application, which is the default Apache web server page. This confirmed that the web application had been completely deleted.

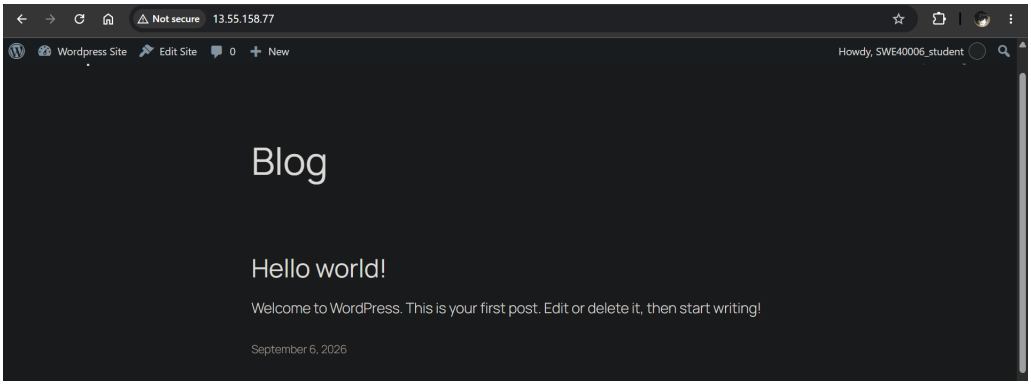

Figure 34: The WordPress application had been deleted.

To restore my deleted web application, I pulled the backup from the S3 bucket and extracted the archived files. The final step was to modify Apache's permissions to be able to read these files and restarting the web server.

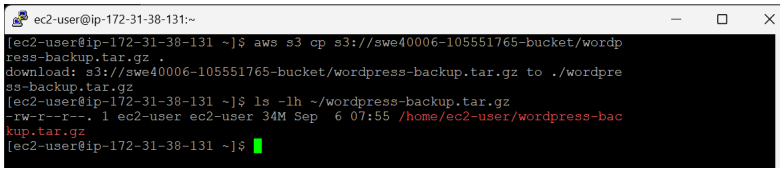

Figure 35: Downloading the backup from the S3 bucket.

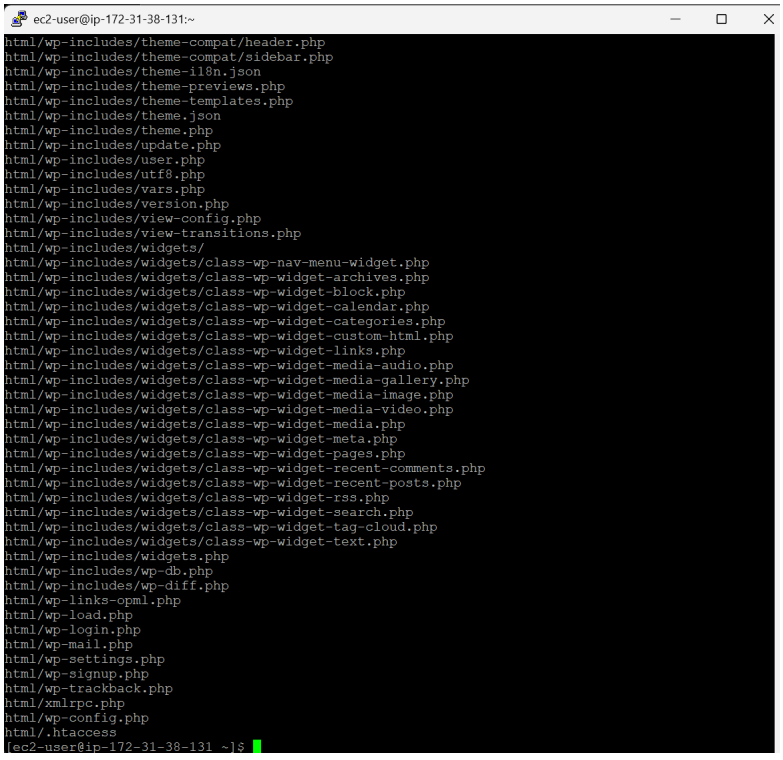

Figure 36: Extracting the web application files.

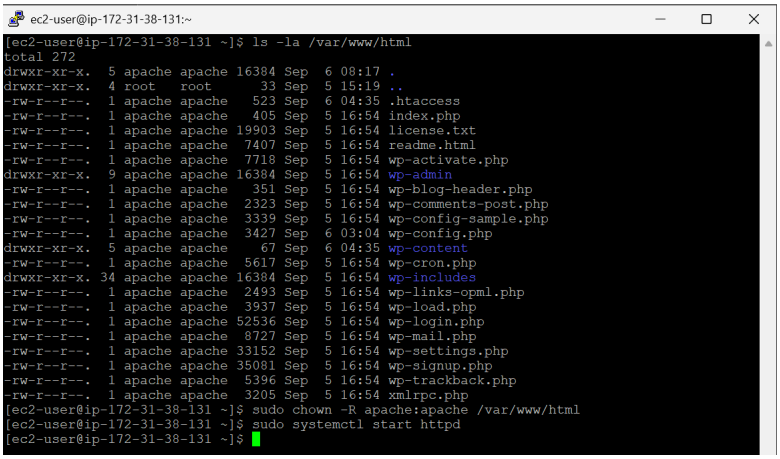

Figure 37: Verifying that the application files have been extracted.

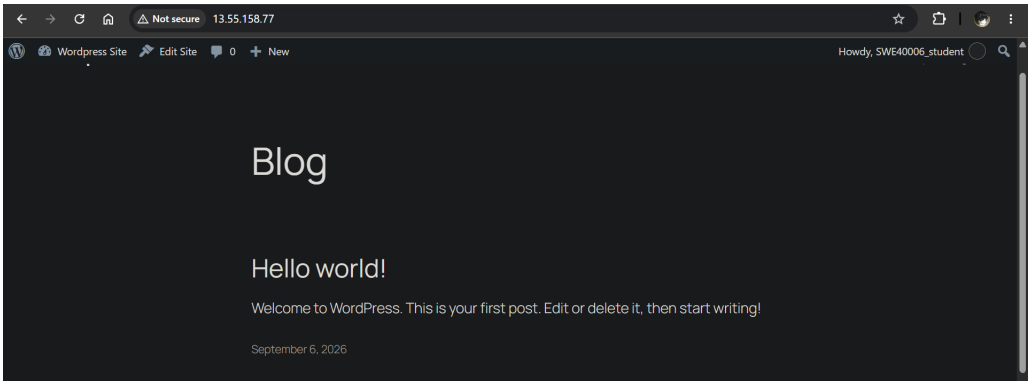

Figure 38: The web application was successfully restored.

2.3 Task 3.3

For this task, I created an EC2 Launch Template containing all the necessary configurations to recreate the web server and application on my original EC2 instance. I also provisioned an Application Load Balancer (ALB) across the `ap-southeast-2a` and `ap-southeast-2b` availability zones to route public HTTP/HTTPS traffic. The load balancer will route traffic to a target group managed by an Auto Scaling Group (ASG). The ASG uses the Launch Template to configure EC2 instances and maintains the desired number of instances, launching new instances in the event of instance failures.

2.3.1 Creating the Launch Template

A new launch template (`task-3-3-template`) is created and configured with the Amazon Linux AMI, the `t3.micro` instance type, and the security group `task-3.1-sg` that was used for the original EC2 instance.

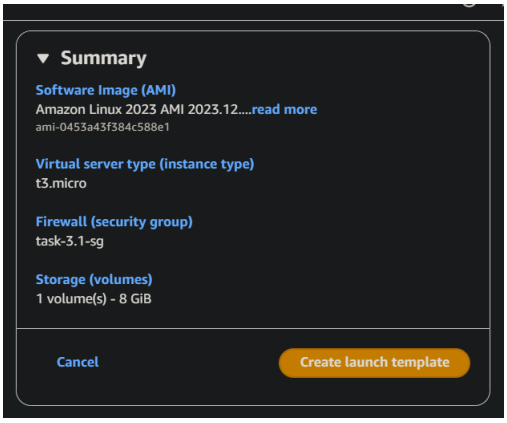

Figure 39: Summary of the launch template's configurations.

Configuring the user data for this template is especially important, as it is the startup script that runs when the instance launches, ensuring that everything necessary is correctly intsalled and configured to serve our web application. The script below updates the OS, installs the web server, PHP, and AWS CLI, starts Apache and PHP_FPM, creates the WordPress web directory, downloads the backup that I created earlier from S3, extracts the files, sets file permissions, and restarts the web server.

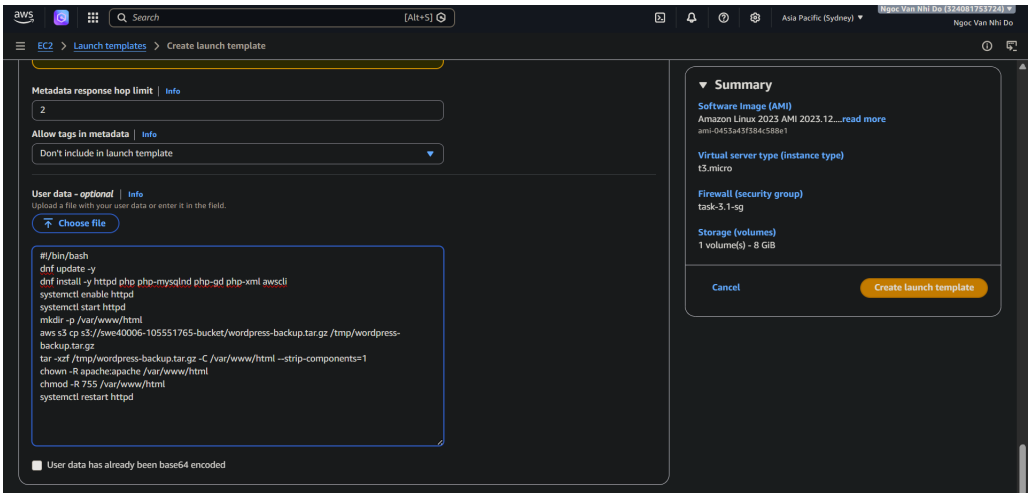

Figure 40: The template's user data script.

2.3.2 Creating the Application Load Balancer (ALB)

Before creating the ALB, I had to set up a new security group for it. The idea is to put the ALB in front my my application instance and make the instances private from the public internet. The ALB will handle incoming HTTP/HTTPS traffic from users on the internet, then distribute these requests across the application instances.

The ALB's security group will allow inbound HTTP/HTTPS traffic from anywhere.

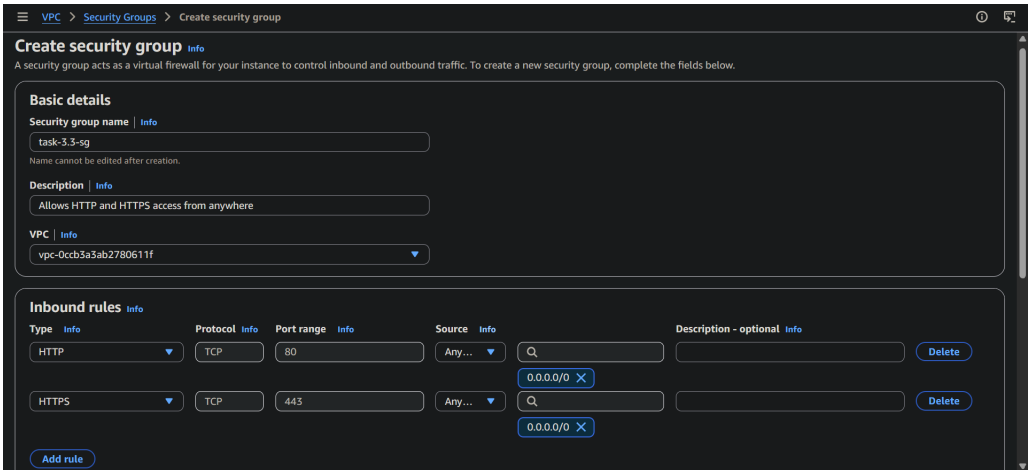

Figure 41: Creating a new security group for the ALB.

The inbound rules for the security group that was used for the original EC2 instance were also changed to only receive HTTP traffic from the ALB's security group. This also implies that the web application hosted on the first EC2 instance is no longer publicly accessible.

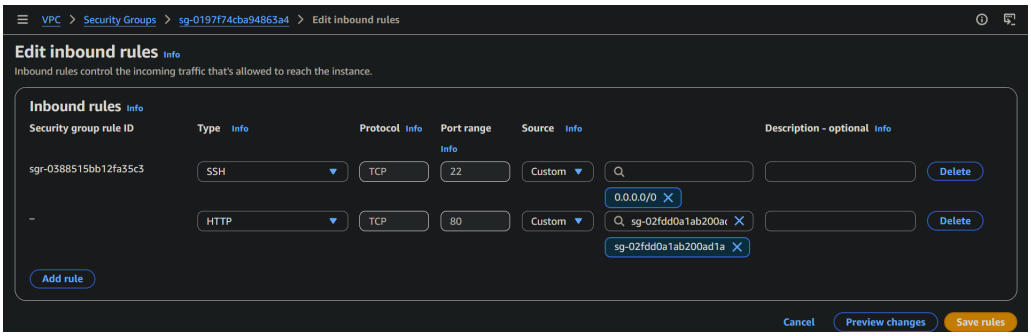

Figure 42: Configuring the rules for the application instances' security group to no longer allow HTTP traffic from the internet.

I then created an ALB (`task-3-3-alb`) spanning the availability zones `ap-southeast-2a` and `ap-southeast-2b`, configured with the security group `task-3.3-sg` that was just created above.

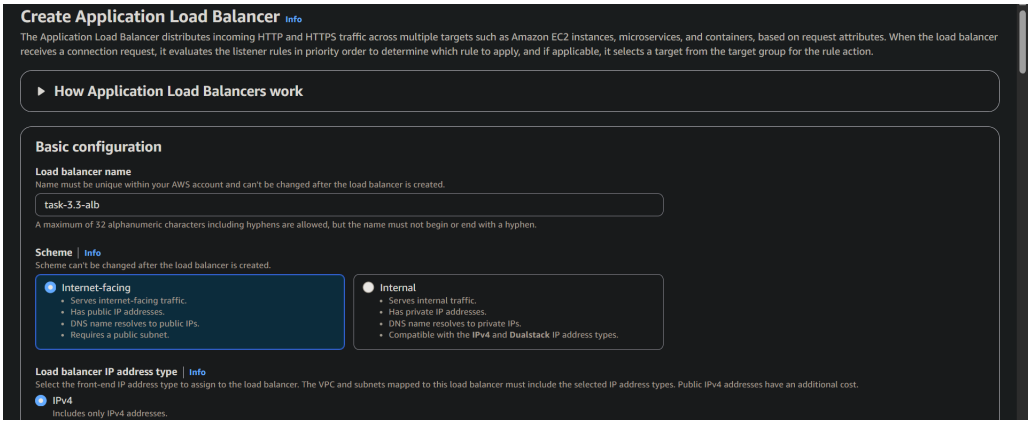

Figure 43: Creating a new ALB with the name `task-3-3-alb`.

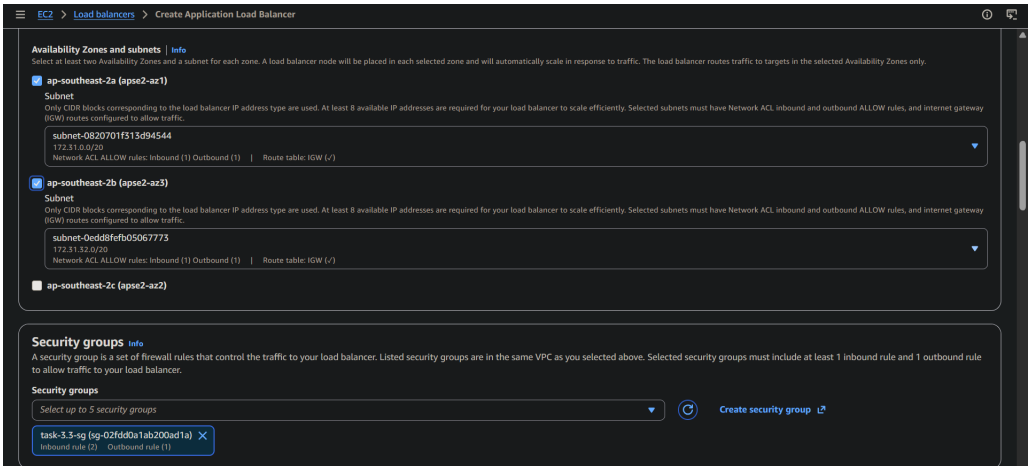

Figure 44: Configuring the ALB's availability zones and security group.

I also created a new target group that would receive traffic from the ALB. A health check was configured on port 80 to verify that the instances would be running and responding correctly. The target group was then attached to the ALB, allowing the ALB to forward incoming traffic to healthy instances.

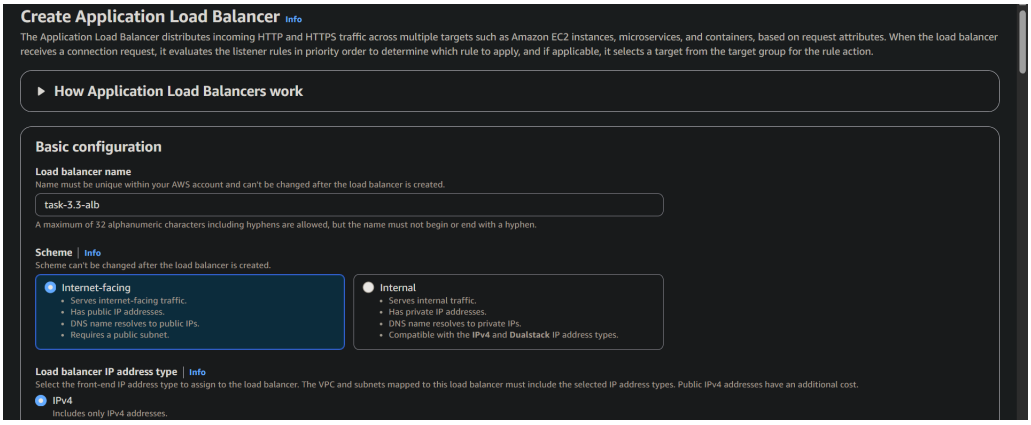

Figure 45: Creating a new target group `task-3-3-tg`. The target group is not registered with any instances for now.

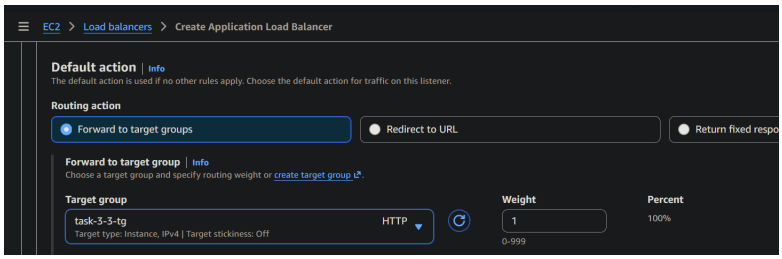

Figure 46: Attaching `task-3-3-tg` to the ALB.

2.3.3 Creating the Auto Scaling Group (ASG)

I then created a new ASG with the following configurations:

- Name: `task-3-3-asg`
- Launch template: `task-3-3-template`
- Availability Zones: `ap-southeast-2a` and `ap-southeast-2b` (matching the ALB's availability zones)
- Load balancer: `task-3-3-alb`
- Target group: `task-3-3-tg`
- Desired capacity: 2
- Minimum capacity: 2

- Maximum capacity: 4

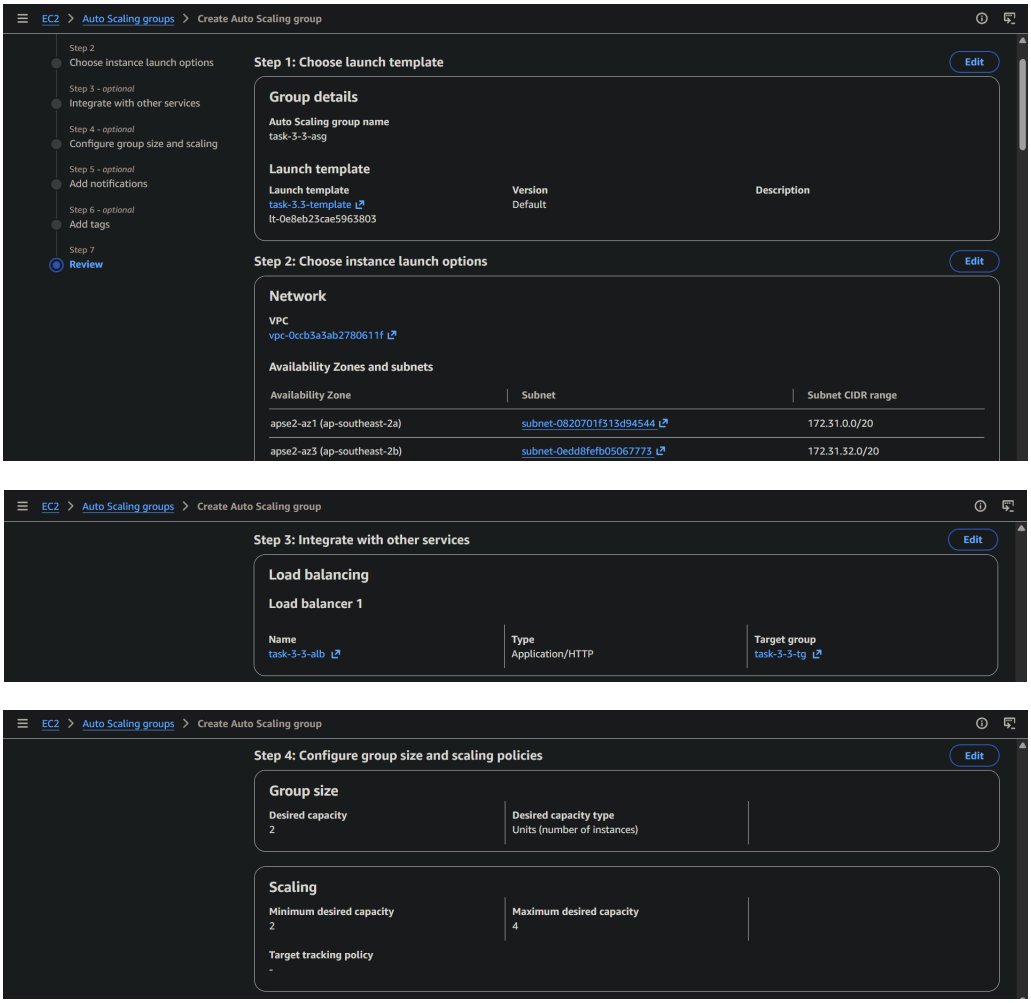

Figure 47: A summary of ASG's configurations.

2.3.4 Verification

Once everything had been set up and configured, I noticed that the health checks for my instances were returning an "Unhealthy" status. I initially suspected that the issue was related to networking for traffic being unable to reach the instances correctly. I began investigating the VPC configuration, security groups, inbound and outbound security rules, and the availability zones to ensure that everything was correctly configured. After confirming that these settings were correct, I decided to SSH into one of the failing instances to further troubleshoot this issue.

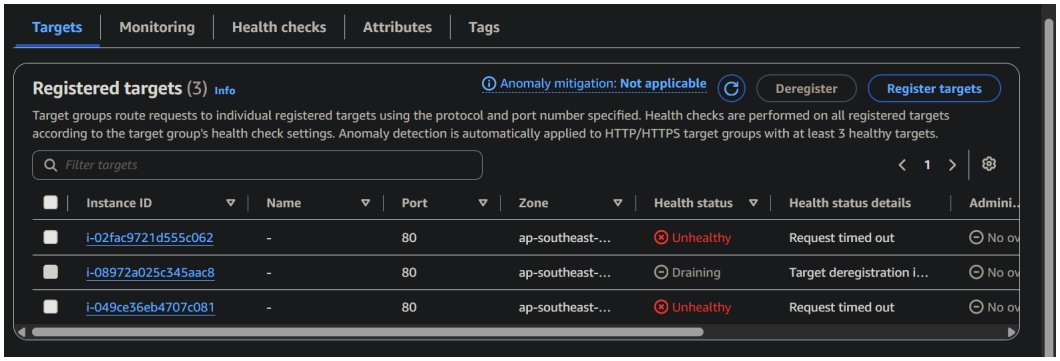

Figure 48: Health checks were returning an "Unhealthy" status, causing the ASG to deregister failing instances and launch new ones to replace them.

I investigated a wide range of potential issues, checking installed dependencies and making sure they were running, verifying that the web application files were properly installed into the correct directory, or that the database configurations contained the correct information. Testing showed that Apache and PHP-FPM were running correctly and that PHP scripts could execute successfully. The WordPress installation was also correctly restored from the S3 backup, including the `wp-config.php` file, which pointed to the RDS database. The root cause was an incorrect RDS security group configuration that prevented the EC2 intsances from connecting to the database on port 3306. When the ALB requests `/`, WordPress tried to connect to the database and hung waiting for the database connection, which made Apache eventually time out.

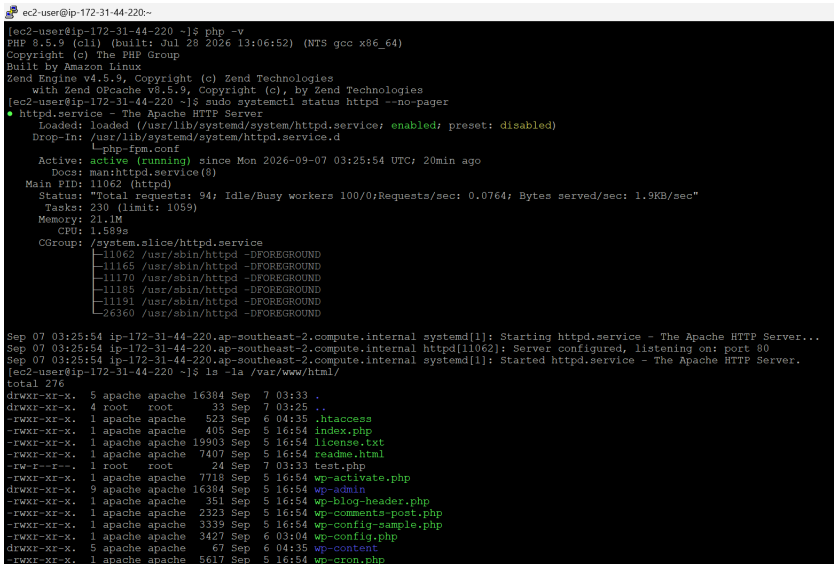

Figure 49: Checking that the application files were restored correctly.

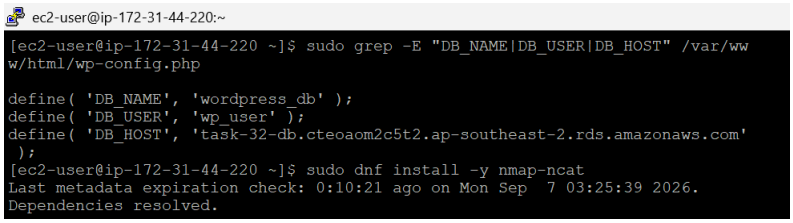

Figure 50: Verifying that the database configurations are correct.

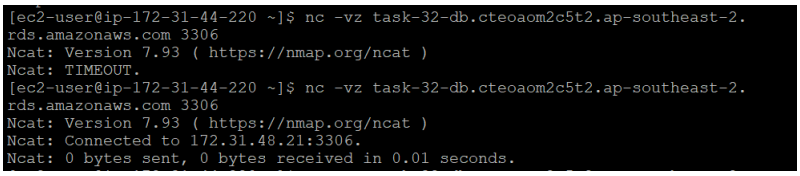

Figure 51: An initial attempt to connect to the RDS database timed out. The second attempt showed a successful connection after the problem had been fixed.

The RDS instance's security group was allowing MySQL traffic on port 3306 from another security group that was not configured with the applications instances. This was fixed by adding the correct inbound security rule.

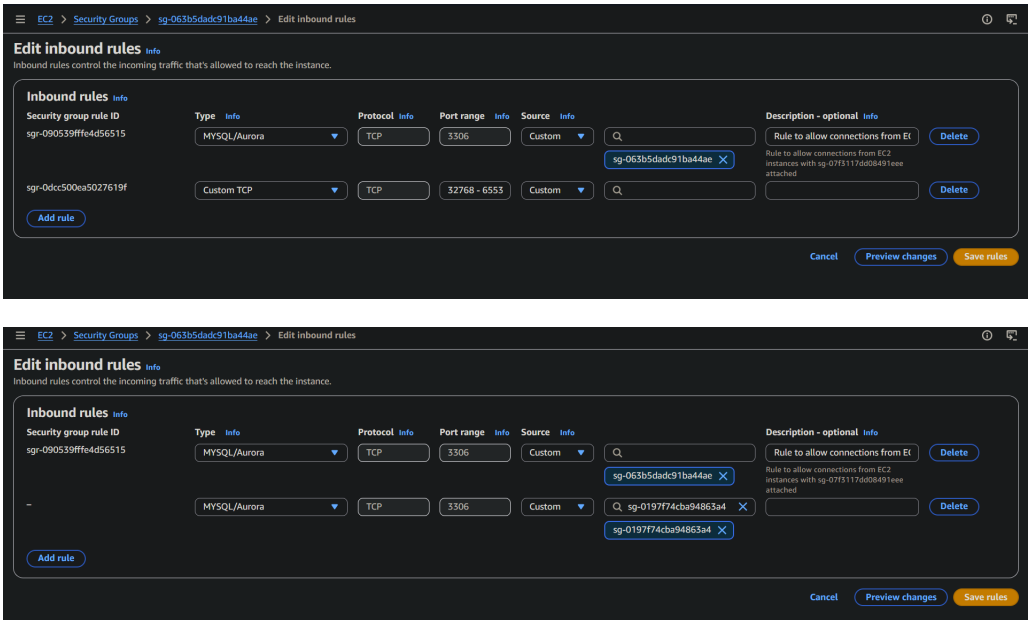

Figure 52: Adding the correct security rule so that the instances could connect to the RDS database.

The health checks now return a "Healthy" status for both instances, confirming that the ALB can communicate with them correctly. The application can be accessed through the ALB's DNS name at http://task-3-3-alb-2099008439.ap-southeast-2.elb.amazonaws.com/.

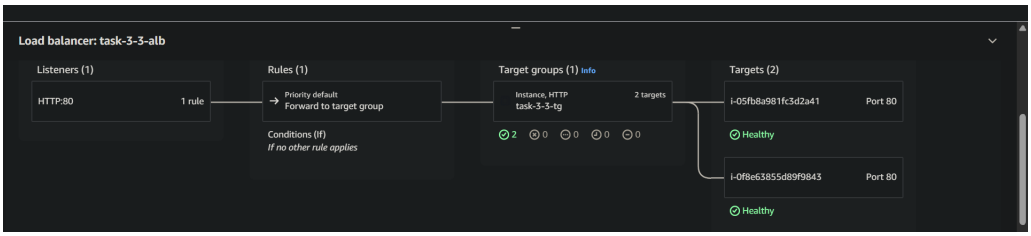

Figure 53: Health checks return "Healthy" status.

Figure 54: The application is running live and can be accessed through the ALB.

2.4 Task 3.4

For task 3.4, I configured a CloudWatch alarm for the Auto Scaling Group to monitor CPU utilisation and trigger when it exceeds 70% over a 5-minute period. An Amazon SNS topic was configured to send an email notification to my student email address when the alarm is triggered. I also enabled AWS Systems Manager (SSM) Session Manager for the EC2 instances, allowing secure terminal access through the AWS Console without requiring public SSH access or open port 22.

2.4.1 CloudWatch Monitoring and Alarm

Before creating the alarm, I set up an SNS topic called `task-3-4-topic`. The CloudWatch alarm uses this topic as the destination to send its notifications, publishing a message when the alarm is triggered by a pre-defined metric. Subscribers to the SNS topic can then receive those notifications.

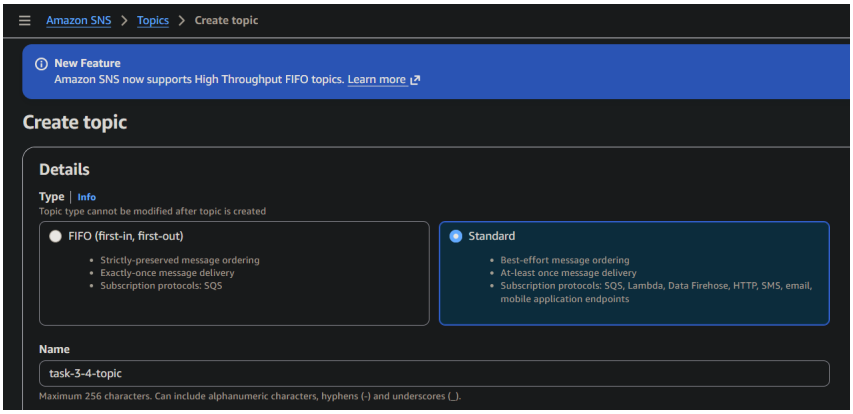

Figure 55: Creating a new SNS topic `task-3-4-topic`.

I then subscribed to this topic so that I could receive an email whenever the CloudWatch alarm gets triggered.

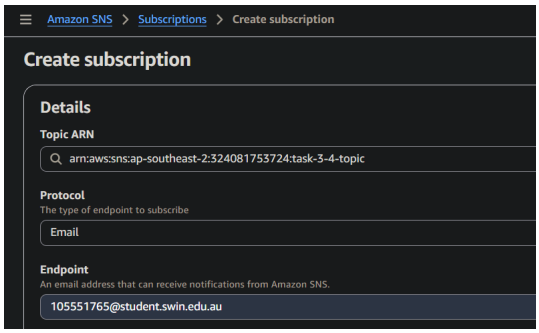

Figure 56: Subscribing to the SNS topic.

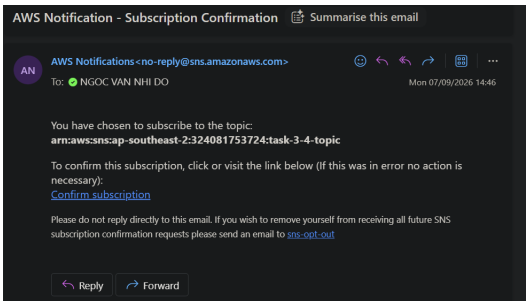

Figure 57: An email is sent to my student email address to verify the subscription.

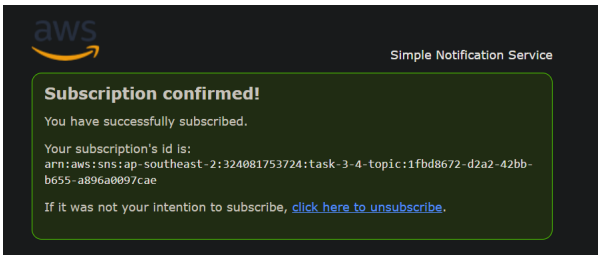

Figure 58: My subscription is verified.

The next step was creating the CloudWatch alarm itself. I set up an alarm on the `CPUUtilization` metric for my ASG `task-3-3` rather on the individual EC2 instances. I also set the threshold for the alarm to "greater than 70", meaning that the alarm is triggered with CPU utilisation exceeds 70% for over 5 minutes.

Figure 59: Creating a new CloudWatch alarm `task-3-4-alarm`.

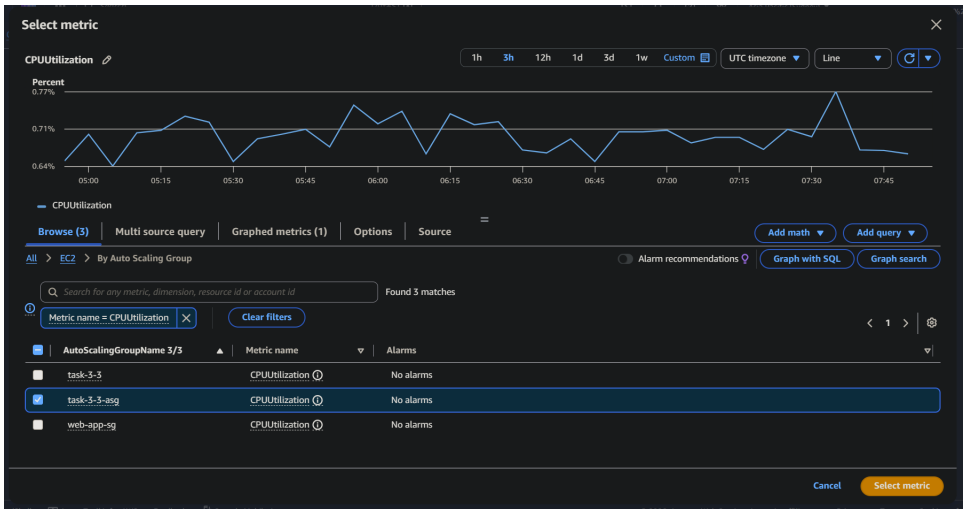

Figure 60: Selecting the `CPUUtilization` metric for my ASG `task-3-3`.

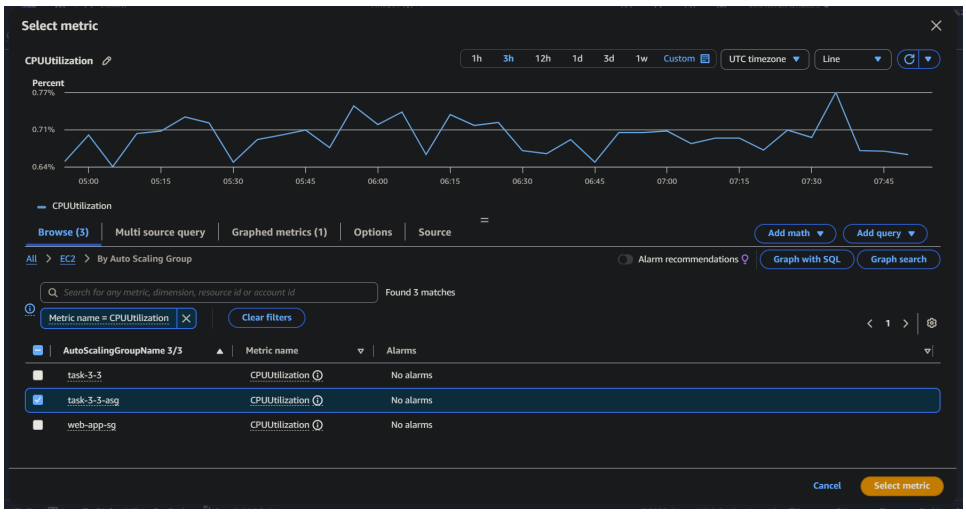

Figure 61: Configuring the threshold for the metric to "greater than 70%".

As previously mentioned, the SNS topic `task-3-4-topic` is attached to the alarm so that I could be notified when the average CPU utilisation metric goes past the defined threshold.

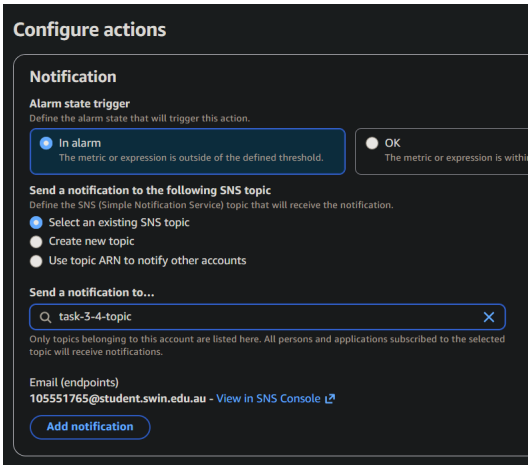

Figure 62: Configuring the metric to Average over a 5-minute period, with the threshold set to Greater than a value of 70.

The alarm has been created. The graph displays the defined threshold as a red dotted line against the current CPU utilisation.

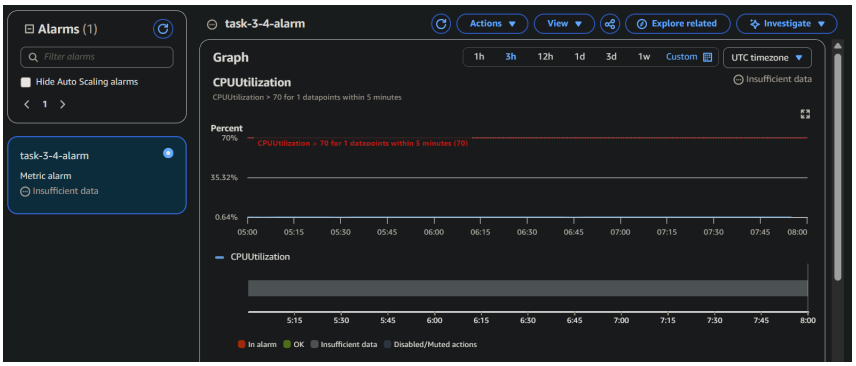

Figure 63: The `task-3-4-alarm` dashboard.

2.4.2 Secure Management via SSM

The goal of this task is to securely access the backend EC2 instances without requiring a private key file or exposting port 22 to the public internet. The first step was to ensure that the EC2 instances had access to the AWS Systems Manager service. This was achieved by attaching the `AmazonSSMManagedInstanceCore` to their IAM role.

I attached this policy to the existing IAM role that was already associated with my EC2 instances.

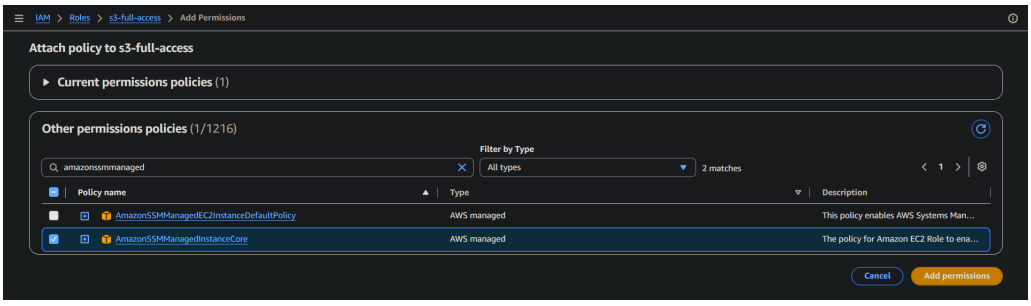

Figure 64: Attaching the policy `AmazonSSMManagedInstanceCore` to the existing IAM role `s3-full-access` (I should've picked a better role name, in retrospect).

As an additional verification step, I SSH-ed into one of the instances to confirm that the SSM agent had already been installed. This is expected, as the SSM agent should be pre-installed on Amazon Linux instances.

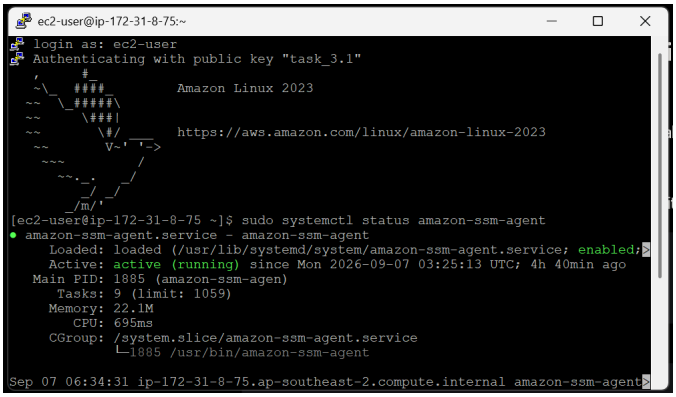

Figure 65: Verifying that the SSM agent already exists on the backend EC2 instances.

Once the role has been modified, the instances are now connected to the Session Manager.

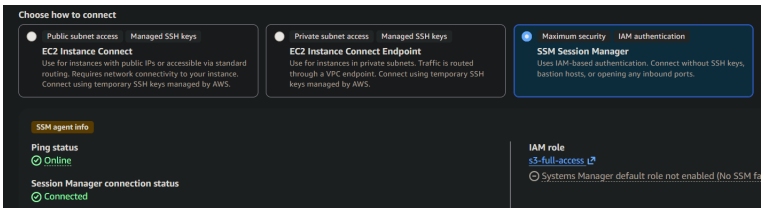

Figure 66: The details of the instance `i-0f8e63855d89f9843` show that it is connected to the Session Manager.

As a proof of concept, I removed the inbound security rule which allowed public SSH access to the EC2 instances. An attempt to SSH into the instance above using PuTTY timed out following this removal.

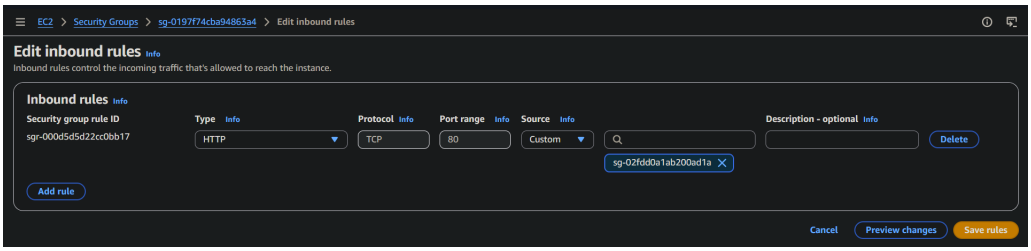

Figure 67: Editing the instances' security group to remove public SSH access.

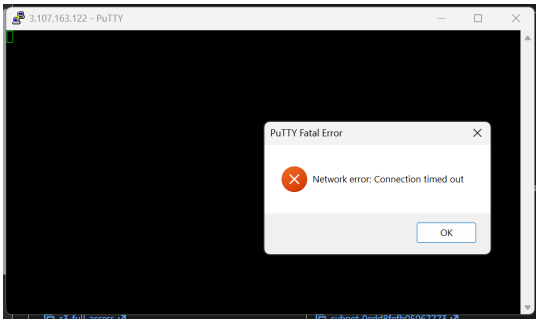

Figure 68: An attempt to SSH into the instance using PuTTY failed.

I then established a connection to the EC2 instance using the Session Manager through the AWS Console to demonstrate that I could securely access the instance's terminal without requiring an SSH connection or the use of a private key.

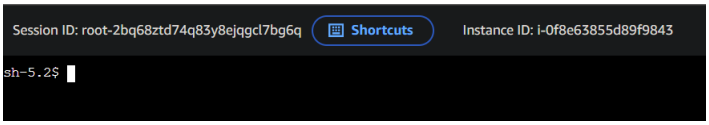

Figure 69: The Session Manager terminal session for the instance `i-0f8e63855d89f9843`.

3 Self-troubleshooting